Readme

📝 项目名称：Auto_prd_test_expert

一、项目结果展示

1. 项目背景与产品定位

1.1 行业痛点

在软件测试领域，测试人员长期面临以下挑战：

- **文档理解耗时**：PRD（需求文档）冗长，人工提取测试点效率低。
- **用例覆盖不全**：容易遗漏边界值、异常场景或安全隐患。
- **经验难以复用**：历史项目的优质用例沉淀在文档中，无法在写新用例时自动联想。
- **AI 生成幻觉**：通用大模型不懂公司内部规范（如密码策略、UI 交互标准），生成的用例“假大空”。

1.2 产品定位

本项目采用 **“轻量化本地部署 + 企业级大模型 API”** 的混合架构，构建了**“精准检索 -> 智能共创 -> 对抗评估”** 的完整闭环体系。该体系不仅能读懂多模态需求进行测试用例生成**，更是具备记忆能力（RAG）和质检能力（AI Critic）**。旨在通过 Human-in-the-Loop（人机协同）模式，解决测试资产生成的准确性、覆盖率及标准化难题，实现企业测试经验的持续沉淀与复用。

2. 核心功能亮点

📚功能一：漏斗式RAG精准检索体系

> **解决痛点**：解决通用大模型不懂具体公司业务规范、PRD文件难以覆盖公司全部背景资料、使用RAG引用内容包含噪音（幻觉）的问题。

- 📥 **输入**：
  - 非结构化的企业资产（技术规范 PDF、历史测试用例库、业务规则文档与图片等）。
  - 当前测试任务的需求描述（PRD）。
- ⚙️ **处理**：
  - **多模态输入处理：**利用Gemini多模态能力进行任意知识文档类型纯文本描述，并生成**简要摘要**与**本地文档存储**，支持**在线预览与删除、本地知识库全量查看**。

- **智能切片**：将上传的技术规范或历史文档按语义进行切分（滑动窗口机制，保留上下文重叠），建立细粒度的向量索引。
- **向量粗筛**：基于 Embedding 向量余弦相似度，从海量知识库中快速召回 Top-K 个相关片段。
- **LLM 智能细筛**：引入轻量级大模型作为“过滤器”，阅读召回的片段，进一步确认信息内容相关度，剔除与当前需求无关的噪音。

- 📥 **输出**：
  - 输出 100% 纯净、高相关的**核心知识上下文**，并向用户**透明展示引用的具体来源文件名**。

🧠功能二：多模态智能共创体系

> **解决痛点**：解决传统工具仅支持文本PRD输入、生成用例后交互体验差无法进行fine-tune的问题。

- 📥 **输入**：
  - 多模态需求材料（拖拽上传 PDF 文档、UI 设计图、补充文本）。
  - 经过功能一清洗后的 RAG 核心知识上下文。
- ⚙️ **处理**：
  - **多模态输入处理**：利用 Gemini多模态能力进行任意PRD文档类型纯文本描述。
  - **双屏共创交互**：采用 IDE 级左右分屏设计。左侧为“思维链与指令区”，**展示 AI 分析测试用例生成思路**，方便用户进行测试大方向把握；**右侧为“实时交付区”**，动态渲染结构化数据，可以进一步细节查看生成的测试用例质量。
  - **多模态上下文迭代：**基于聊天窗口的**上下文记忆能力**，支持**追加上传新文档/图片或输入自然语言指令**，对大模型输出结果进行持续**微调 (Fine-tune)** 与纠偏。
- 📤 **结果**：
  - 实时生成结构化测试用例，支持Excel, YAML, Markdown, JSON 四种格式的一键下载。
  - **用例回流**：支持将最终确认的**用例回档**至本地**RAG**知识库，自动转化为下一次生成的“历史参考案例”，实现系统的自我进化。

⚖️功能三：智能对抗评估体系

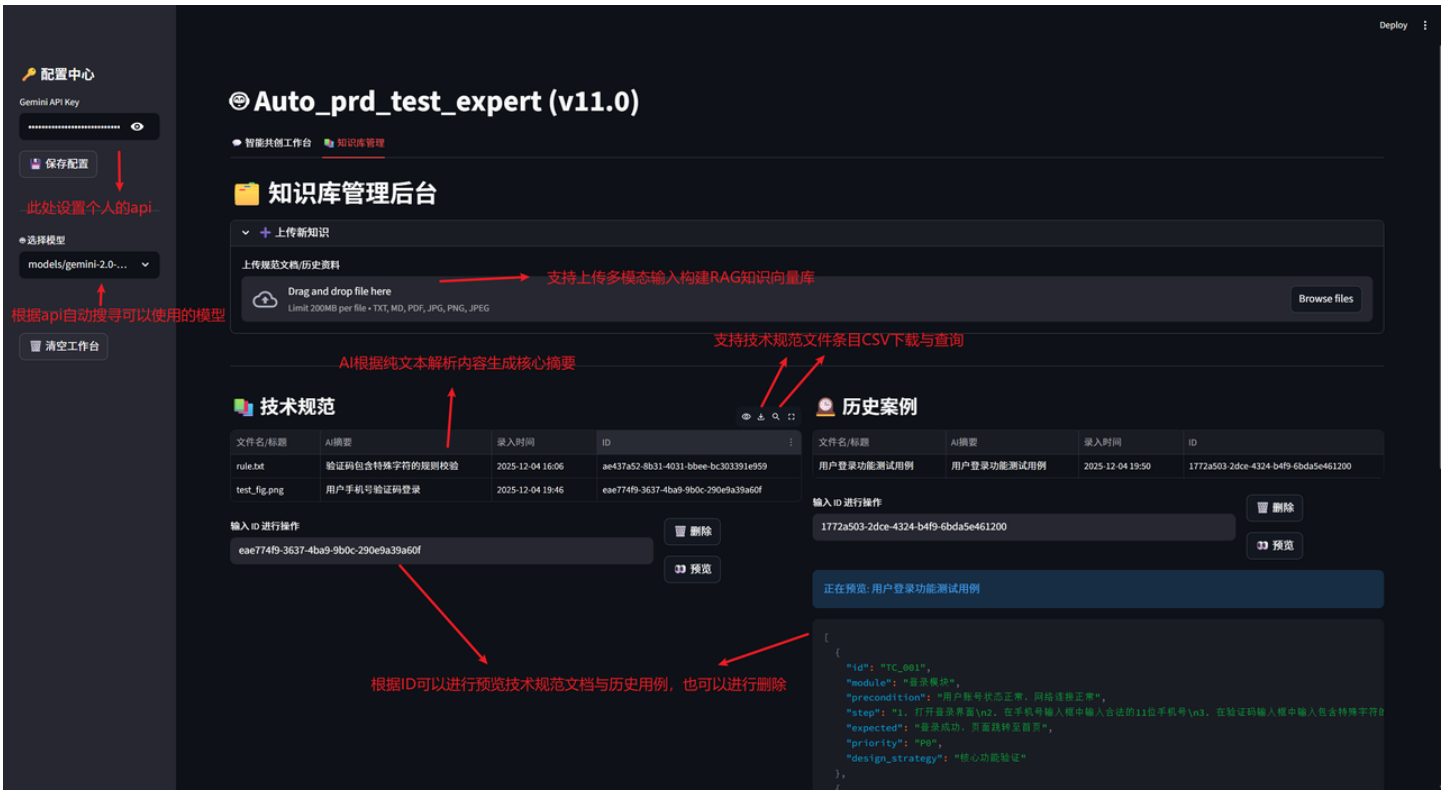
> **解决痛点**：解决人工验收效率低、AI 生成用例逻辑自洽性差、缺乏标准度量的问题。

- 📥 **输入**：
  - 功能二生成的待评审用例。
  - 原始 PRD 文档及 RAG 规范。
  - （可选）人工上传的标准参考用例。
- ⚙️ **处理**：

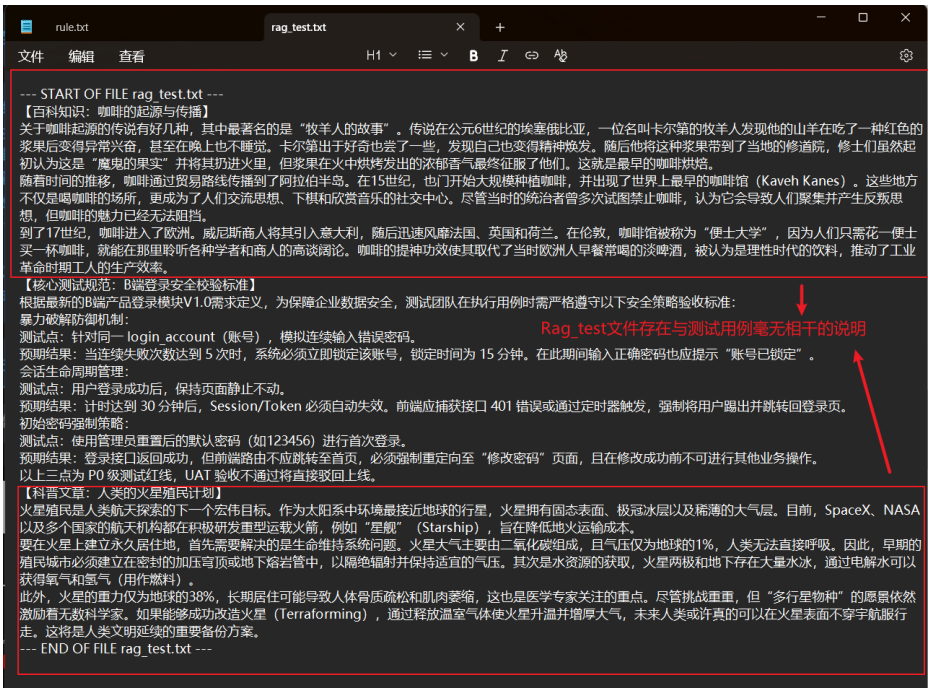
- **独立评审 Agent (AI Critic)**：引入 Multi-Agent 协作思想，构建独立的“QA 验收专家”角色，与“生成者”形成对抗。
- **多维质检算法**：
  - **覆盖率分析：反向溯源 PRD与RAG文档**，检测是否有未覆盖的功能点。
  - **逻辑自洽性检查**：检测操作步骤 (Step) 与预期结果 (Expected) 是否存在因果矛盾。
  - **去重与规范性**：自动检测逻辑重复的冗余用例。
- **差异化比对**：若存在标准用例，自动计算生成结果与标准答案的偏差值。

- 📤 **结果**：
  - 输出一份可视化的**质量评估报告**，包含健康度评分、问题清单（漏测/错误/重复）及具体的优化建议，辅助用户进行最终决策。

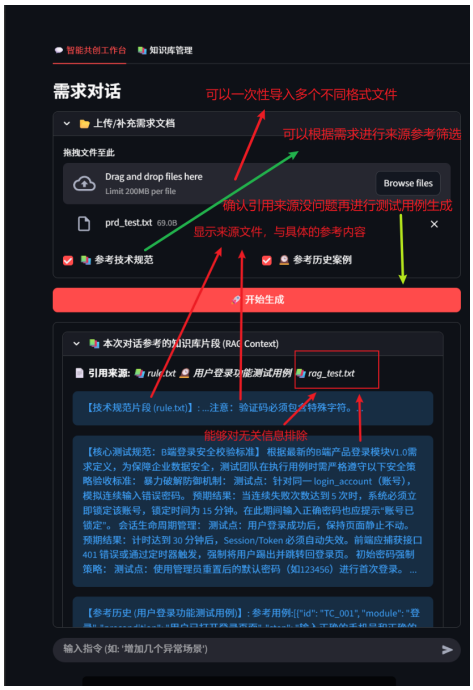
Demo展示：



图一 上传多模态文件进行RAG向量库生成与UI前端可视化预览，搜寻、删除、下载与核心摘要生成



(a)



(b)

图二 构建RAG知识库的污染文件与利用RAG知识库与LLM进行相关知识检索生成

(a)

(b)

(c)

(d)

(e)

图三 进行测试用例生成后人工微调、查看、多种格式下载、入库

图四 测试专家对测试用例评估效果展示

3. 技术栈总结与选型理由

3.1 核心开发环境

- **Python 3.10**
- **Anaconda**: 进行不同项目python版本控制
- **Git**：进行代码迭代版本控制

3.2 前端交互层 (Frontend)

- **Streamlit**:

- **选型理由**: 专为数据科学设计的 Web 框架，支持快速构建响应式界面。
- **关键应用**: 利用 `Session State` 维护复杂的对话上下文和文件上传状态；使用 `st.columns` 实现左右分屏布局。

3.3 后端逻辑层 (Backend)

- **Google Generative AI SDK**:
  - **选型理由**: 原生支持 Gemini api，具备超长上下文（1M+ tokens）和原生多模态能力。
- **Pandas**: 负责 JSON 数据到结构化表格的清洗与转换，支持 Excel 导出。

3.4 RAG 数据层 (Data & Retrieval)

- **ChromaDB**:
  - **选型理由**: 轻量级、本地化的向量数据库，无需服务器维护，数据存储在本地 `data/vector_db` ，保障**数据隐私安全**。
- **Embedding Model**: 使用 Google `text-embedding-004` ，兼顾性能与免费额度。

4. 核心代码模块说明

本项目采用模块化分层架构，各层职责清晰，通过标准接口进行数据交互：

📂 配置层 (`config/`)

- `prompts.py` **(提示词管理)**:
  - **功能实现**: 该模块通过 Python 类集中管理生成、评估、清洗及摘要的所有 Prompt 模板。它利用**字符串格式化**技术，实现了**智能路由逻辑**：根据前端传递的上下文自动判断用户意图（闲聊 vs 任务），动态组装 System Prompt。同时，在 Prompt 中强制约束了 **“解释（Thought）与数据（JSON）分离”** 的输出格式，为后端的数据清洗提供了标准化依据。

📂 核心逻辑层 (`core/`)

- `rag_engine.py` **(知识引擎)**:
  - **功能实现**: 这是 RAG 架构的核心载体。它封装了 **ChromaDB** 向量数据库的 CRUD 操作，实现了基于滑窗算法的递归文本切片器 (`TextSplitter`)，确保向量索引的颗粒度。
  - **技术应用**: 在多模态解析上，直接调用 **Gemini API** 将上传的 UI 图片/PDF 转化为文本描述，实现了非结构化数据的统一向量化。同时优化了 `get_doc_content` 方法，优先从向量库 Metadata 读取解析后的文本，解决了前端无法直接预览二进制文件的技术难题。
- `llm_client.py` **(AI 网关)**:
  - **功能实现**: 负责与大模型进行通讯。它封装了 **Google Generative AI SDK**，管理与 LLM 的会话生命周期。

- **技术应用**: 实现了带有历史记忆的 `get_gemini_chat_response` 接口，将前端的 `st.session_state` 历史记录转换为 SDK 标准格式，支持流式对话。同时包含动态模型获取逻辑，根据 API Key 权限自动适配 Flash/Pro 等不同版本模型。

- `evaluator.py` **(评估引擎)**:
  - **功能实现**: 独立的质量质检模块。它接收原始 PRD、RAG 上下文及生成的用例，构建特定的评估 Prompt。
  - **技术应用**: 调用 LLM 进行逻辑推理，并将非结构化的评语强制解析为标准的 **JSON 格式**（包含分数、问题列表），供前端渲染为可视化仪表盘。

📂 前端交互层 (`ui/`)

- `main.py` **(主控台)**:
  - **功能实现**: 作为系统的神经中枢，基于 **Streamlit** 框架构建，全权负责用户交互流、Session 状态管理（如文件流、对话历史 `messages`、评估报告 `eval_report`）以及各子模块的调度。
  - **技术应用 & Prompt 调用点**:
    i. **RAG 智能双重筛选**: 在知识库上传背景文档阶段，触发 `MULTIMODAL_PARSE_PROMPT` 与 `SUMMARY_PROMPT`，调用大模型进行多模态文档识别，并自动生成精炼的业务标题，并存入 ChromaDB；在PRD文件上传时首先调用 `rag_engine` 进行向量粗筛；随即调用 `llm_client` 并加载 `RAG_FILTER_PROMPT`，让大模型充当“质检员”，对召回的切片进行去噪和提纯，确保上下文的高相关性。

    ii. **思维链共创对话**: 在左侧聊天区，通过 `get_gemini_chat_response` 接口，根据用户意图（生成 vs 闲聊）动态加载 `CORE_SYSTEM_PROMPT`（含意图路由逻辑）。对于生成任务，它会结合 `INITIAL_PROMPT` (初次生成) 或 `REFINEMENT_PROMPT` (后续微调) 驱动大模型进行逻辑推导，并利用 `split_text_and_json` 函数实现“分析思路”与“结构化数据”的视觉分离。

    iii. **智能对抗评估**: 在右侧评估 Tab 中，响应用户点击，调用 `evaluator` 模块并加载 `EVALUATOR_SYSTEM_PROMPT`，驱动大模型扮演 QA 角色对生成的用例进行多维质检（覆盖率、逻辑性、去重）。

    iv. **资产归档与摘要**: 在用户确认归档时，触发 `SUMMARY_PROMPT`，调用大模型为本次生成的测试用例自动生成精炼的业务标题，并存入 ChromaDB 形成闭环。

- `components.py` **(组件库)**:
  - **功能实现**: 专注于数据的可视化呈现与导出。
  - **技术应用**: 利用 **Pandas** 将 JSON 数据转化为结构化的 DataFrame 进行表格渲染。集成了多种序列化库，实现了测试用例向 **CSV, JSON, YAML** 等格式的一键转换与下载，确保交付物可直接导入下游测试管理平台。

- `sidebar.py` **(配置中控台)**:
  - **功能实现**: 作为全局配置入口，负责管理 API Key 的输入与持久化存储，以及会话状态的重置。
  - **技术应用**: 实现了**动态模型加载逻辑**，它调用后端 `llm_client` 的接口获取当前 Key 可用的模型列表，并利用 **Streamlit** 的 `selectbox` 组件渲染下拉菜单，实现了配置的灵活性与容错性。

二、未来展望

未来计划在以下三个维度进行深化：

1. 输入端：接口化与多态拓展

- **Web/IM 集成**：开发 FastAPI 接口层，脱离 Streamlit 限制。支持通过 **飞书/钉钉机器人** 接收需求文档，直接在 IM 群组中返回测试用例，实现“对话即测试”。
- **URL 解析**：增加对网页链接（如在线 PRD、Jira 单据链接）的直接解析能力。

2. 输出端：自动化测试集成 (Agent化)

- **自动化脚本生成**：
  - 利用生成的结构化数据（YAML/JSON），进一步调用大模型生成 **Pytest** 或 **Midscene.js** 测试脚本。
  - 实现从“生成文档”到“生成可执行代码”的质变。
- **自动执行与报告**：集成 CI/CD 流水线，自动运行生成的脚本，并将测试报告回传至前端，并入**RAG 进行历史回流存档**。

3. 上游优化：PRD 智能完善

- **需求反向优化**：增加“PRD 质检”模块。在生成用例前，先对用户输入的简略 PRD 进行分析，指出逻辑漏洞或模糊地带，并利用 AI 自动扩写和丰富需求文档。